What Makes Binary Search Fast on a Desktop CPU? Branchless, Prefetched, Eytzinger and Interleaved Searches from L1 to DRAM

Attila Torda

October 2026

Abstract

We measure 22 implementations of searching a sorted array of 64-bit integers on an AMD Zen+ desktop processor, for arrays from 8 KiB to 512 MiB, with two compilers, both for independent lookups (throughput) and for lookups that depend on each other (latency). The study started as a check of our own earlier benchmark, which had reported hand-written assembly and AVX2 searches beating the textbook search by up to 31%. That benchmark drew its keys with `rand()`, which returns at most 32,767 on both Windows C libraries, so on large arrays it only ever searched the first 256 KiB: it reported 147 ns per search where a correct key distribution gives 650 ns. With keys spread over the whole array, none of the earlier assembly or SIMD variants beats the C reference by much, and the textbook "branchless" search becomes *slower* than the branchy one on arrays that do not fit in the caches. What works is memory-level parallelism. Interleaving 16 independent searches in lockstep is 4.31× faster than the reference at 512 MiB and 4.42× faster in L1. For a single search, the Eytzinger layout with prefetching cuts latency by 2.33× at 512 MiB. Two smaller findings matter in practice: power-of-two array sizes cost up to 1.79× more than their neighbours, and whether a compiler turns a select into a conditional move or a branch can change a variant's speed by a factor of three. We describe the harness, the pitfalls, and the variants we adopted in an open-source library.

1 Introduction

Binary search over a sorted array is among the most used algorithms there are: it backs dictionary lookups in databases, symbol tables, sorted-set membership, and the merge and join steps of query engines. Its comparison count, $\lceil \log_2(n+1) \rceil$, is optimal. On a modern processor that count says little about its speed. Each step loads an element whose address depends on the previous comparison, so on an array larger than the caches every step can be a cache miss, and the steps form a dependency chain that the processor cannot overlap. A branchy implementation adds a branch misprediction on half of all steps.

Many variants address one or the other cost: branch-free ("branchless") updates, software prefetching of the next probe, cache-friendlier layouts such as Eytzinger (breadth-first) order, B-tree-like blocking and SIMD comparisons, and processing several searches at once. Khuong and Morin's study [5] is the standard reference; it found that on large arrays the Eytzinger layout with prefetching is usually fastest, and that for small arrays a good branchless binary search on the sorted array is best.

This paper reports a smaller, practical study with three aims. First, we correct our own earlier measurements (Section 5): a key generator that looked harmless confined every search to the first

256 KiB of the array, and the published conclusions about large arrays were artifacts. The pitfall is easy to fall into and invisible in the results, which makes it worth documenting. Second, we measure the earlier hand-written assembly and AVX2 variants against the established alternatives across the whole memory hierarchy of one desktop processor, with two compilers and in both a throughput and a latency setting (Sections 4 and 6). Third, we turn the results into the defaults of an open-source C library (Section 7).

2 Background

The machine. All measurements use an AMD Ryzen 5 2600X (Zen+, six cores, 3.6 GHz base clock, 4.25 GHz boost) with 16 GB of dual-channel DDR4 running at 2133 MT/s, under Windows 10 22H2 with the "High performance" power plan. Each core has a 32 KiB L1 data cache and a 512 KiB L2 cache; the 16 MiB L3 is split into two 8 MiB halves, one per core complex, so a single thread sees 8 MiB of L3 [1]. The L1 data TLB has 64 entries and the L2 TLB 1,536 entries for 4 KiB pages, covering 6 MiB. The core can have up to 22 L1 data-cache misses in flight [1]. Our arrays use 8-byte keys, so the L1, L2 and L3 boundaries fall at 2^{12}, 2^{16} and 2^{20} elements.

Variants of binary search. A *branchy* search compares the key with the middle element and branches; the branch is unpredictable, but the processor speculates past it and starts the next load early, which on a cache miss acts as a prefetch of one of the two possible next probes. A *branchless* search [5] replaces the branch with a conditional move: no mispredictions, but also no speculation, so every step waits for its load. Adding *software prefetches* of both possible next probes recovers the overlap at the cost of extra memory traffic. The *Eytzinger* layout stores the implicit binary search tree in breadth-first order, so the next probes of the next several steps share a cache line and can be prefetched together [5]. Cache-oblivious layouts [2] and SIMD-friendly k-ary layouts [6, 10] change the layout too; we do not include them, because our library must search arrays in the caller's sorted order. Finally, when there are many keys to look up, the searches can be *interleaved*: several are advanced one step each in turn, so their cache misses overlap. Group prefetching for hash joins [3], asynchronous memory access chaining [7] and coroutine-based interleaving of index lookups [9] apply this idea to database operators.

3 Variants

Table 1 lists what we measured. All variants search the same sorted array in place, except Eytzinger, which needs its own copy of the data in breadth-first order; the time to build that copy is measured separately. All variants that return an exact match return the same index (the keys are distinct), which lets every timed run be checked with a checksum.

The branchless lower bound is the formulation of Khuong and Morin [5]:

```
const int64_t* base = a;
while (n > 1) {
    int64_t half = n / 2;
    base = (base[half] < key) ? base + half : base;   /* cmov */
    n -= half;
}
return (base - a) + (*base < key);
```

Table 1: The variants. *Prototype* variants come from our earlier study, unchanged; the others are written in portable C. "Calls" are single-key functions called through a function pointer, as a library caller would; "batch" variants take an array of keys.

Variant	Description
Reference	textbook branchy search with early exit on equality (C)
`std::lower_bound`	C++ standard library, then one equality test
Prototype asm, branching	the reference in NASM assembly, Win64 calling convention
Prototype asm, cmov	assembly with `cmov` bound updates and an early-exit branch
Prototype asm, "pipelined"	assembly that loads the next middle element at the end of each step
Prototype AVX2	four probes per step, compared with one AVX2 instruction
Prototype AVX2 + prefetch	the same, with `prefetcht0` of two guessed addresses
Branchless	lower bound with a conditional select and a fixed trip count [5]
Branchless + prefetch	the same, prefetching both possible next probes every step
Eytzinger + prefetch	breadth-first layout, branchless, prefetching three levels ahead [5]
Interleaved, K keys	$K \in \{4, 8, 16, 32\}$ branchless searches advanced in lockstep
+ prefetch	each key's next probe prefetched as soon as it is known
select	the first version of the interleaved kernel (pointer select, see Section 6.5)

The trip count depends only on n, so all searches over the same array take the same number of steps. Interleaving exploits this: K searches share the loop and its counter, and only their positions differ. In our final kernel the step is written as integer arithmetic on a position, not as a pointer select:

```
pos[j] += half & -(int64_t)(a[pos[j] + half] < key[j]);
```

Section 6.5 explains why.

4 Method

Data and keys. An array of n elements holds $0, 2, 4, \ldots, 2(n-1)$. Each run draws 2^{19} keys from a SplitMix64 generator [11]: a uniform index $i \in [0, n)$ and a coin, giving the key $2i$ (present) or $2i + 1$ (absent). Every search thus targets a uniformly random position of the whole array, and half of the searches fail.

Sizes. n runs from 2^{10} to 2^{26} (8 KiB to 512 MiB) in half-octave steps; sizes between powers of two are rounded to integers.

Modes. In *throughput* mode the keys are independent, so the processor may overlap consecutive searches, as in a loop over a list of lookups. In *latency* mode each key is combined with the previous result before the search, $k'_i = k_i \oplus (r_{i-1} \wedge z)$, where z is read from a `volatile` variable that holds zero: the key is unchanged, but the compiler and the processor must wait for the previous search to finish. This models a chain of dependent lookups, such as following references. Interleaved variants exist only in throughput mode.

Timing. One pass searches all 2^{19} keys and is timed with `std::chrono::steady_clock`. Each variant runs one untimed pass, then nine timed passes; the passes of all variants are interleaved round-robin, so that slow periods of the machine affect all variants alike. We report the median and

the interquartile range of the nine passes in nanoseconds per search. The thread is pinned to one core and runs at high priority.

Correctness. Before timing, each variant is compared with the reference on the four edge keys (below the minimum, the minimum, the maximum and above it) and on the first 4,096 keys of the run. Every timed pass also sums the returned indices, and the sum must equal the reference's. Every measurement reported here passed both checks.

Repeatability. Within a run, the nine passes agree closely: the median interquartile range is 7% (GCC) and 1% (MSVC) of the median. Between runs they agree less, for arrays between 1 and 16 MiB, i.e. from twice the L2 cache to twice the L3 a core sees. Repeating that range (2^{17} to 2^{21} elements) in a second run gave times that differ from the first by a median factor of 1.07 (GCC) and 1.20 (MSVC), and by up to 1.48× and 2.68×, not always in the same direction for all variants. Each run allocates its arrays afresh, so the likely cause is the placement of the array's physical pages, which decides how the array maps onto the cache sets in that range. At the two ends of the range the runs agree within 18% (GCC) and 28% (MSVC); we did not repeat the other sizes. Both runs are in the published results; comparisons between variants within one run are reliable throughout, comparisons between runs (such as between compilers) only outside 1–16 MiB.

Compilers. MinGW-w64 GCC 13.2 with `-O3` and Microsoft Visual C++ 19.41 (Visual Studio 2022, toolset 14.41) with `/O2`, both without `-march`/`/arch` options; the AVX2 prototypes carry a function-level target attribute. Single-key variants are called through a function pointer, which models a call into a library and prevents inlining into the timing loop.

5 The pitfall: keys that never leave the cache

Our earlier benchmark generated keys as

```
srand(12345);
keys[i] = (rand() % size) * 2;          /* or * 2 + 1 for a miss */
```

`RAND_MAX` is 32,767 in both the Microsoft and the MinGW C runtime, so `rand() % size` never exceeds 32,767. For arrays of up to 32,768 elements nothing is wrong. For larger arrays, every search targets the first 256 KiB, which stays in the L2 cache, and the first comparisons of every search go the same way and are perfectly predicted. The earlier benchmark also used only 1,000 distinct keys for large arrays. Its conclusion that "at 10 million elements the array exceeds the L3 cache and the search is memory-bound" was therefore not tested by the experiment that seemed to support it.

Figure 1 repeats the measurement with the old generator and with uniform keys. With the old keys the time per search barely grows beyond 32,768 elements; the reference search at 2^{24} elements takes 147 ns, while uniform keys give 650 ns, 4.43× more. The error also changes rankings: with the old keys the prototype's branchless assembly looks competitive at large sizes (156 ns), while with uniform keys it is the slowest variant (1178 ns). Nothing in the old results looked suspicious; times grew slowly with the array size, as one might expect from a logarithmic algorithm.

Two habits would have caught it: drawing keys from a generator whose range is known to cover the index space, and checking that the time per search for an array far larger than the last-level cache is close to the memory latency times the number of steps that miss. Measurement artifacts of this kind are a known hazard [4, 8]; this one needs no exotic cause.

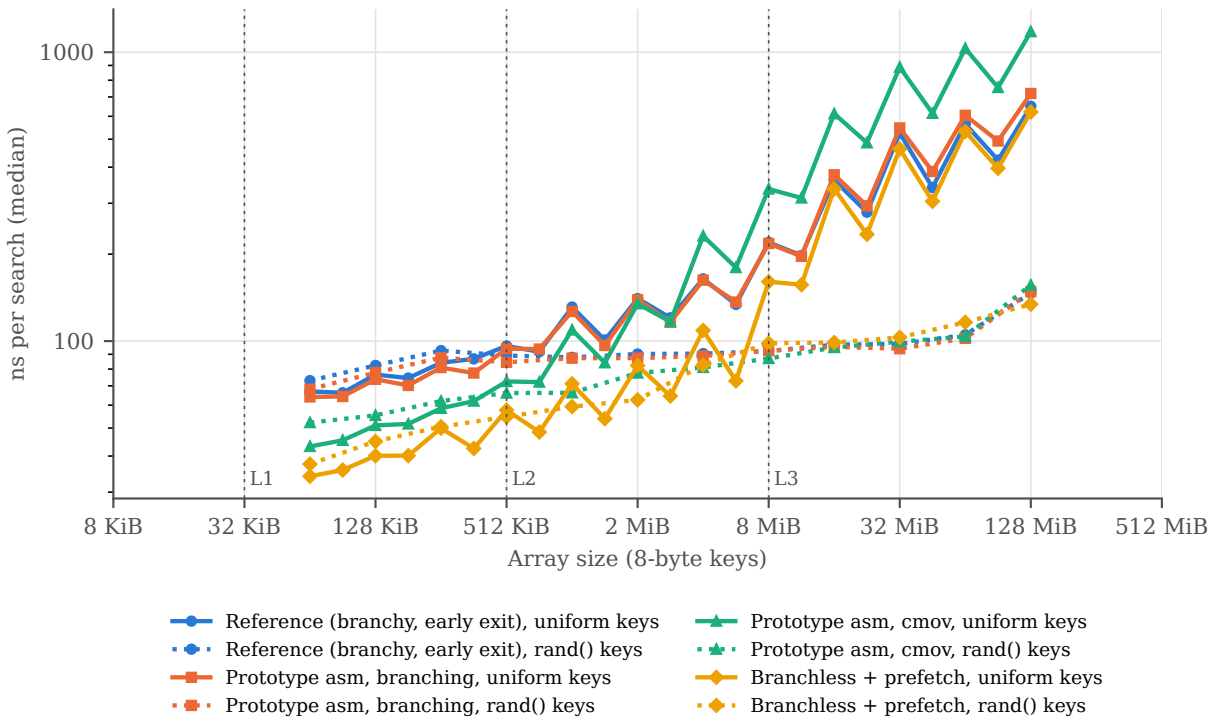


Figure 1: The same variants with the earlier benchmark's `rand()` keys (dotted) and with keys uniform over the array (solid), GCC, throughput mode. Beyond 2^{15} elements (256 KiB) the old keys never leave the first 256 KiB.

6 Results

Figures 2 and 3 show the main variants compiled with GCC; Table 2 lists all variants at five sizes, one in each level of the memory hierarchy. The MSVC results are in Table 3 and discussed in Section 6.5.

6.1 Single searches

In the caches, the branchless search is the fastest search over the sorted array: 25.3 ns at 32 KiB against 57.4 ns for the reference (2.27×). Every branchy variant, including `std::lower_bound` and the hand-written assembly, pays for a misprediction on about half of its steps; their curves lie on top of each other.

Beyond the caches the order reverses. At 512 MiB the branchless search takes 1021 ns, more than the reference's 830 ns. The branchy search mispredicts, but while it waits for a load it speculatively issues the load of one of the two next probes; half of the time it guessed right and that load is already on its way. The branchless search issues nothing until its comparison resolves. Prefetching both candidate probes restores the overlap: the branchless search with prefetching is 1.09× faster than the reference at 512 MiB and 1.91× faster in L1. In the caches the two extra prefetch instructions per step cost it up to 1.26× the time of the plain branchless search; beyond L3 they cut its time to as little as 0.60×. This reproduces Khuong and Morin's finding [5] on a different microarchitecture.

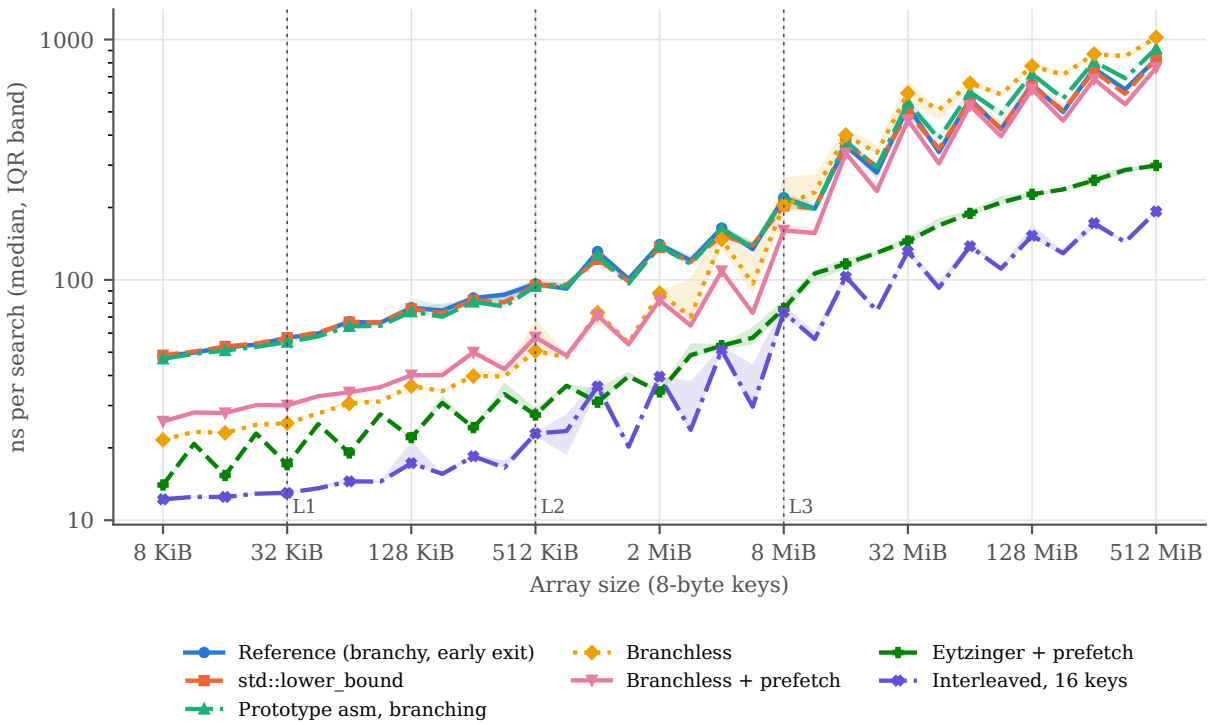


Figure 2: Throughput mode, GCC: nanoseconds per search, median of nine passes of 2^{19} searches, with the interquartile range shaded. Dotted lines mark the capacity of L1, L2 and the L3 a single core sees.

Eytzinger order is the fastest single search from L2 outwards. Its prefetch fetches the cache line holding all eight nodes three levels below the current one, so the loads of the next three steps are already under way: at 512 MiB it is 2.78× faster than the reference in throughput mode and 2.33× in latency mode. The price is a second copy of the data in a different order, built in a few nanoseconds per element, and a result that is a position in that copy, not in the sorted array.

The latency mode removes the overlap between consecutive searches. The branchless search suffers most: 1306 ns at 512 MiB against 1021 ns in throughput mode; in throughput mode the out-of-order core had been running the next search's first steps during the current search's misses. The reference changes little, and Eytzinger stays the fastest.

6.2 Interleaved searches

Interleaving is the largest single improvement in this study. With 16 keys in flight, a search costs 193 ns at 512 MiB, 4.31× less than the reference and 1.55× less than Eytzinger; in L1 it is 4.42× faster than the reference. The width matters (Figure 4). At 512 MiB a search costs 467 ns with four keys in flight, 276 ns with eight, 193 ns with sixteen and 209 ns with 32: beyond sixteen there is nothing more to gain, consistent with the 22 outstanding L1 misses the core can track. Explicit prefetching of each key's next probe, which is essential for single searches, does not help interleaved searches: with 16 independent loads per step the core already has enough misses in flight, and the prefetch instructions only add work.

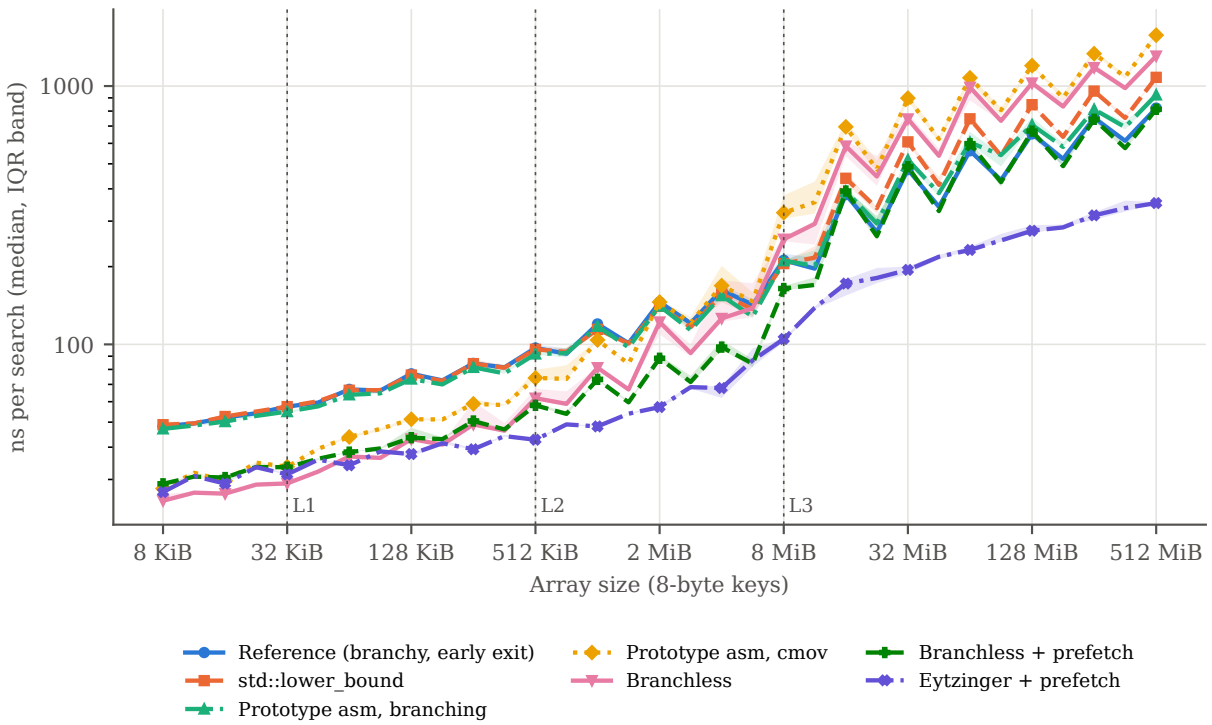


Figure 3: Latency mode, GCC: every search depends on the result of the previous one.

6.3 The prototype's assembly and SIMD variants

None of the earlier hand-written variants is worth keeping. The branching assembly performs like the C reference, as it should: it is the same algorithm. The "pipelined" assembly, which loads the next middle element at the end of each step, performs the same, because that load still depends on the comparison just made. The `cmov` assembly keeps an early-exit branch on equality and has no prefetch, so it combines the drawbacks of both styles and is the slowest variant beyond L3 (1498 ns at 512 MiB). The AVX2 variants compare four probes at once, but the four probes are scattered over the array and must be loaded one by one, and they shrink the interval to a quarter per step instead of a half. Compiled with GCC they are never faster than the reference (at best 1.00× its time); they only beat MSVC's reference on large arrays, where that reference is slow for the reason given in Section 6.5.

6.4 Power-of-two sizes

The zigzag in Figures 2 and 3 is real and reproducible. Searches over arrays of exactly 2^k elements are slower than over arrays of $2^{k\pm1/2}$ elements: by up to 1.70× for the reference and 1.79× for the interleaved search, measured against the geometric mean of the two neighbouring sizes. In an array of 2^k elements the first probes sit at offsets $2^{k-1}, 2^{k-2}, \ldots$ elements, whose addresses agree in their low bits and therefore map to the same cache sets; the frequently used top of the implicit search tree competes for a few sets of an 8-way L1 and L2. The Eytzinger layout stores the top of the tree contiguously and shows no such penalty on large arrays (1.02×); on small arrays it is even faster at powers of two (0.70×), because then nearly every search takes the same number of steps and the

Table 2: GCC 13.2, nanoseconds per search (median). Columns are array sizes in L1, L2, L3 and DRAM; the fastest variant per column is in bold.

	Throughput (ns/search)					Latency (ns/search)				
Variant	32 KiB	512 KiB	8 MiB	64 MiB	512 MiB	32 KiB	512 KiB	8 MiB	64 MiB	512 MiB
Reference (branchy, early exit)	57.4	96.2	220	565	830	57.4	96.8	212	562	821
`std::lower_bound`	57.4	94.7	203	552	832	57.4	95.7	205	747	1081
Prototype asm, branching	55.1	94.1	218	604	918	54.9	91.8	211	605	924
Prototype asm, cmov	33.2	72.5	336	1031	1498	33.6	74.2	323	1076	1574
Prototype asm, “pipelined”	52.3	94.4	199	603	902	52.5	90.7	214	617	885
Prototype AVX2	59.2	102	251	642	959	59.4	102	252	654	919
Prototype AVX2 + prefetch	58.6	112	265	741	1088	58.6	108	283	735	1096
Branchless	25.3	50.6	204	658	1021	**29.0**	61.9	255	989	1306
Branchless + prefetch	30.1	57.7	161	531	764	33.4	58.1	164	598	813
Eytzinger + prefetch	17.0	27.4	75.8	189	299	31.4	**42.7**	**105**	**232**	**352**
Interleaved, 8 keys	**11.7**	23.3	87.3	200	276	–	–	–	–	–
Interleaved, 16 keys	13.0	22.9	**73.3**	**138**	**193**	–	–	–	–	–
Interleaved, 32 keys	18.3	27.9	78.7	149	209	–	–	–	–	–
Library (single key)	26.9	57.2	187	565	809	30.3	58.2	187	603	858
Library (batch)	13.4	**22.2**	86.9	141	221	–	–	–	–	–

loop exit is predicted. Benchmarks that only use powers of two overstate the cost of classic binary search; ours report both.

6.5 Compilers and if-conversion

Whether a compiler turns a conditional into a branch or a conditional move (“if-conversion”) changes the performance of these kernels more than any other compiler difference we saw (Figure 5). The two compilers were measured in separate runs, so between 1 and 16 MiB the figure shows the run-to-run variation described in Section 4 rather than a compiler effect; the effects below lie outside that range.

The reference search. MSVC compiles the reference’s two bound updates into `cmovge` instructions; GCC keeps a branch. MSVC’s reference is therefore a branchless search without prefetching, with exactly that profile: faster than GCC’s in the caches (0.71× GCC’s time at 32 KiB) and much slower beyond L3 (1.90× at 128 MiB). The same C source thus has opposite performance characteristics depending on the compiler, and neither is best everywhere.

The interleaved kernel. Our first interleaved kernel updated a pointer per key with a conditional select, like the branchless single search. GCC unrolls the loop over keys for 16 keys and emits conditional moves; for 32 keys it does not unroll, keeps the pointers in memory and emits a conditional branch, which mispredicts on half of all steps. That version took 59.5 ns per search in L1, against 18.3 ns for the final kernel, which computes the step as `half & -(a[pos + half] < key)`: there is no select left to turn into a branch. MSVC compiled both versions without branches. Code that depends on if-conversion for its performance should not leave the choice to the compiler’s unrolling heuristics.

7 What went into the library

The SpeedUp library previously routed every search to the reference. It now offers:

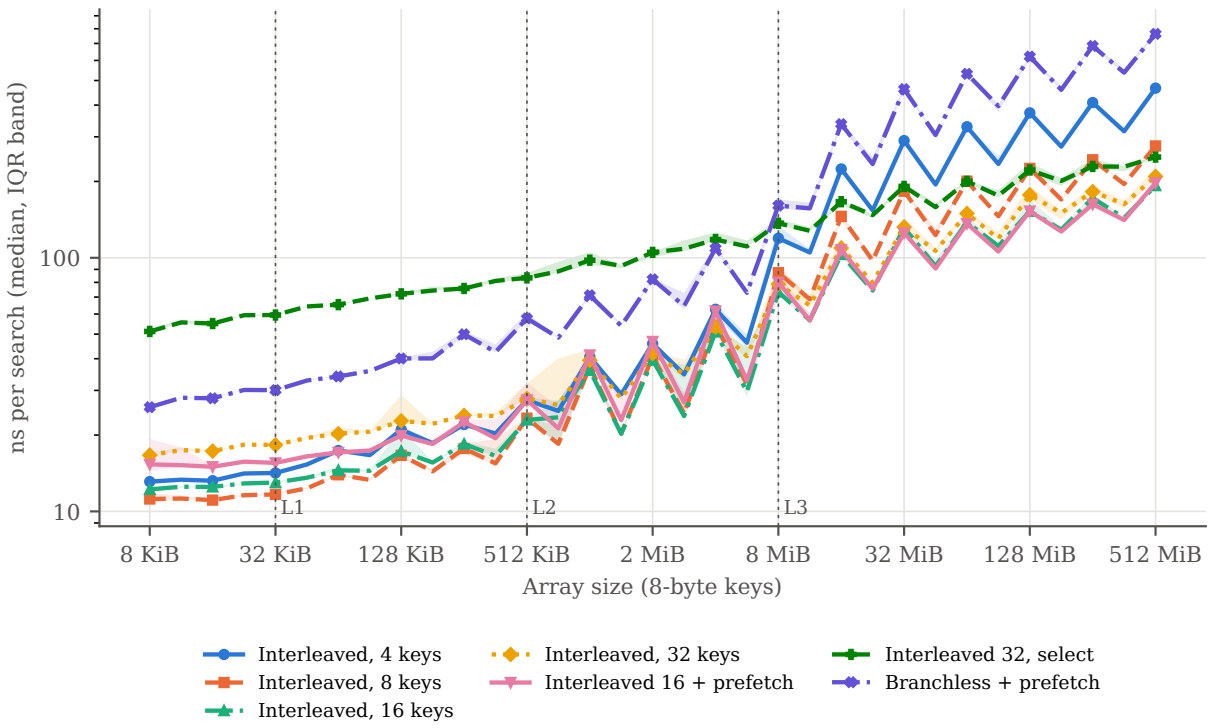


Figure 4: Interleaved searches by width, GCC, throughput mode, with the single-key branchless search with prefetching for comparison.

- `speedup_binary_search_i64` and `speedup_lower_bound_i64`: the plain branchless search while the array fits the L2 size given by the library's cache hint (256 KiB by default), and the branchless search with prefetching beyond it. Among all thresholds, switching at the L2 size cost the least against the better of the two kernels at each size, over both compilers and both modes; always prefetching would cost up to a quarter more in the caches. With GCC the library takes at most 1.01× the reference's time in throughput mode and 1.07× in latency mode, at any size, and down to 0.47× in the caches; with MSVC at most 0.71×. Eytzinger order would be faster on large arrays, but it requires the caller to store the data in that order.
- `speedup_binary_search_batch_i64` and `speedup_lower_bound_batch_i64`: the interleaved kernel with 16 keys, without prefetching, in the mask formulation. For callers that have many keys at once, this is the fastest search of a sorted array that we measured, at every size and with both compilers; only Eytzinger order, which reorders the data, beats it at a few sizes in L2.

The library rows of Tables 2 and 3 were measured after this change, in a separate run with the same settings. The exact-match functions return the first occurrence of the key, which the reference did not guarantee. All entry points are checked against a linear scan in an exhaustive test (every key and gap for all sizes up to 1,025, arrays with duplicates, extreme values, and three arrays of up to four million elements).

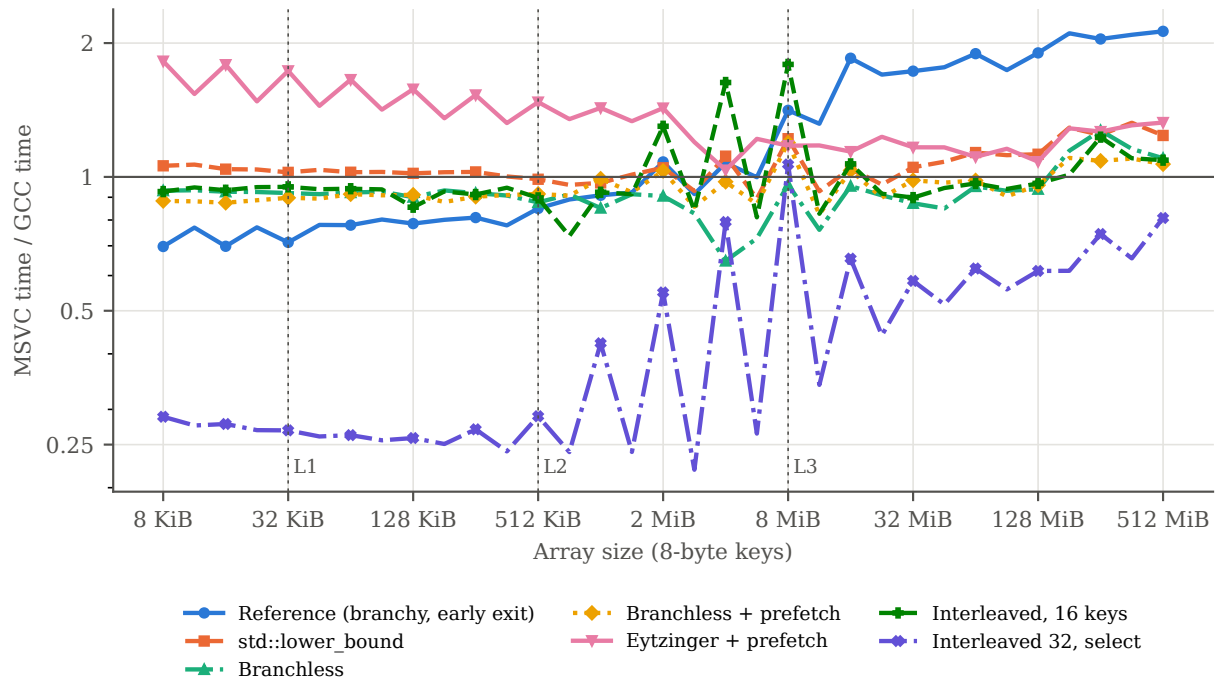


Figure 5: MSVC time divided by GCC time, throughput mode; above 1, GCC's code is faster.

8 Threats to validity

All results come from one processor, one memory configuration and one operating system. Zen+ has a comparatively small number of outstanding misses and a split L3; processors with more memory-level parallelism will favour interleaving and prefetching even more, and processors with a larger unified last-level cache will shift the crossover points. The memory ran at 2133 MT/s, below the modules' rating, which raises absolute DRAM latencies. We used 4 KiB pages; huge pages would remove most TLB misses on large arrays and reduce all large-array times, and they might also remove the run-to-run variation between 1 and 16 MiB. Absolute times in that range should be read as one sample of a distribution over memory placements; a study of that range should repeat each run with fresh allocations. The keys are uniform; skewed or sorted key streams make every variant faster and favour branchy code. Finally, each variant is one implementation, and other formulations may compile differently, as Section 6.5 shows they can.

9 Conclusion

On arrays that fit in the caches, binary search is limited by branch mispredictions and a branchless search wins. On larger arrays it is limited by memory latency, and only techniques that keep several cache misses in flight help: prefetching both candidates of the next step, the Eytzinger layout, or, best of all, interleaving many independent searches. Hand-written assembly and SIMD comparisons of scattered probes did not help. The most consequential error in our earlier study was not in any search but in the key generator, and the most surprising effects were outside the algorithms too: power-of-two sizes and compiler if-conversion each changed results by a factor of up to two or three.

Table 3: MSVC 19.41, nanoseconds per search (median); compare Table 2.

	Throughput (ns/search)					Latency (ns/search)				
Variant	32 KiB	512 KiB	8 MiB	64 MiB	512 MiB	32 KiB	512 KiB	8 MiB	64 MiB	512 MiB
Reference (branchy, early exit)	40.9	81.7	310	1069	1765	41.2	82.3	320	1070	1984
`std::lower_bound`	58.8	93.6	248	627	1032	59.1	93.6	247	623	1373
Prototype asm, branching	55.9	89.9	263	592	1014	55.4	90.0	255	599	1269
Prototype asm, cmov	33.6	69.3	311	1012	1682	33.8	70.2	274	1014	2023
Prototype asm, “pipelined”	52.7	86.6	258	600	1058	52.3	86.5	255	599	1292
Prototype AVX2	62.1	98.7	275	685	1189	62.4	99.6	278	688	1309
Prototype AVX2 + prefetch	60.2	99.1	304	769	1315	60.3	99.2	306	778	1431
Branchless	23.3	44.3	196	626	1125	**26.8**	54.7	229	838	1728
Branchless + prefetch	27.0	52.8	192	522	816	30.1	53.6	189	565	1028
Eytzinger + prefetch	29.5	40.4	89.1	210	397	32.7	**43.7**	**97.5**	**220**	**444**
Interleaved, 8 keys	**11.7**	21.1	130	196	291	–	–	–	–	–
Interleaved, 16 keys	12.3	**20.6**	131	**133**	**210**	–	–	–	–	–
Interleaved, 32 keys	14.5	23.5	134	136	211	–	–	–	–	–
Library (single key)	25.9	58.1	183	575	856	30.3	56.8	211	611	896
Library (batch)	12.8	28.7	**77.6**	144	214	–	–	–	–	–

Availability

The library, the benchmark harness, the raw results and the scripts that produce every figure, table and number in this paper are at `https://github.com/attilatorda/SpeedUp` (AGPL-3.0):

- harness: `benchmarks/binary_search/bench_bsearch.cpp`;
- results: `benchmarks/outputs/bsearch_ryzen2600x_*.csv`;
- figures, tables and numbers: `benchmarks/scripts/run_bsearch_benchmarks.py`.

References

[1] 7-cpu.com. AMD Zen: cache, TLB and latency figures. `https://7-cpu.com/cpu/Zen.html`. Accessed October 2026.

[2] Gerth Stølting Brodal, Rolf Fagerberg, and Riko Jacob. Cache oblivious search trees via binary trees of small height. In *Proceedings of the 13th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 39–48, 2002.

[3] Shimin Chen, Anastassia Ailamaki, Phillip B. Gibbons, and Todd C. Mowry. Improving hash join performance through prefetching. In *Proceedings of the 20th International Conference on Data Engineering (ICDE)*, pages 116–127, 2004.

[4] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC15)*, pages 73:1–73:12, 2015. doi: 10.1145/2807591.2807644.

[5] Paul-Virak Khuong and Pat Morin. Array layouts for comparison-based searching. *ACM Journal of Experimental Algorithmics*, 22:Article 1.3, 2017. arXiv:1509.05053.

[6] Changkyu Kim, Jatin Chhugani, Nadathur Satish, Eric Sedlar, Anthony D. Nguyen, Tim Kaldewey, Victor W. Lee, Scott A. Brandt, and Pradeep Dubey. FAST: Fast architecture

sensitive tree search on modern CPUs and GPUs. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, pages 339–350, 2010. doi: 10.1145/1807167.1807206.

[7] Onur Kocberber, Babak Falsafi, and Boris Grot. Asynchronous memory access chaining. *Proceedings of the VLDB Endowment*, 9(4):252–263, 2015.

[8] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[9] Georgios Psaropoulos, Thomas Legler, Norman May, and Anastasia Ailamaki. Interleaving with coroutines: A practical approach for robust index joins. *Proceedings of the VLDB Endowment*, 11(2):230–242, 2017. doi: 10.14778/3149193.3149202.

[10] Benjamin Schlegel, Rainer Gemulla, and Wolfgang Lehner. k-ary search on modern processors. In *Proceedings of the Fifth International Workshop on Data Management on New Hardware (DaMoN)*, pages 52–60, 2009.

[11] Guy L. Steele, Jr., Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 453–472, 2014. doi: 10.1145/2660193.2660195.